# Hybrid Search Engine for Programming Questions:

## Bridging the Vocabulary Gap via Dense, Lexical Retrieval and Reranking

Tivdzua Lubem Noah Sikeh Gisele Wiykiynyuy Venera Bikbulatova

April 22, 2026

**Abstract**

Programming-question retrieval is difficult because of a persistent vocabulary gap between how users describe problems and how relevant technical discussions are phrased. A user may search with an informal symptom such as *website is white after running javascript*, while the most useful Stack Overflow thread may be framed in terms of a precise exception, API error, or implementation detail. This project investigates whether a hybrid retrieval pipeline can better address that mismatch than lexical or semantic retrieval alone.

We implement and evaluate four systems on the MTEB StackOverflowDupQuestions benchmark: BM25, Dense Retrieval, Hybrid Reciprocal Rank Fusion (RRF), and Hybrid + Cross-Encoder Reranking. The benchmark test split contains 89,470 documents and 2,992 queries. Precision@10, Recall@10, MAP, and nDCG@10 are used for evaluation. To ensure fair MAP comparison across systems, all systems are evaluated at a shared ranking depth of 20 for MAP, while Precision@10, Recall@10, and nDCG@10 remain measured at cutoff 10.

The strongest overall system is Hybrid (RRF), which achieves Precision@10 = 0.721491, Recall@10 = 0.241327, MAP = 0.350161, and nDCG@10 = 0.747357. A one-sided Wilcoxon signed-rank test on per-query nDCG@10 shows that both Hybrid (RRF) and the best reranked system significantly outperform BM25, with all reported p-values below $10^{-6}$. A rerank-depth ablation further shows that reranking the top-20 fused candidates outperforms reranking deeper candidate pools. Overall, the results show that combining lexical and semantic retrieval through RRF is the most effective strategy for reducing the vocabulary gap in this technical search setting.

## 1 Introduction

Technical forums such as Stack Overflow are central to software problem solving, but retrieving the right prior discussion is often difficult. The main challenge is a **vocabulary gap**: the wording used by a user at search time can differ substantially from the wording used in the relevant duplicate thread.

A beginner may describe a problem through symptoms, for example a white page, a failing script, or unexpected behavior, while the most relevant technical discussion may instead contain a specific exception name, framework term, or API description. Purely lexical retrieval is strong

when exact terms overlap, but it struggles when users and relevant documents use different vocabulary. Purely semantic retrieval can bridge that mismatch more effectively, but it may underweight exact technical tokens that are crucial in programming search.

This project studies whether a hybrid retrieval strategy can combine the strengths of both approaches. We implement and compare four systems:

1. **BM25** lexical retrieval
2. **Dense retrieval** with a bi-encoder and FAISS
3. **Hybrid (RRF)** fusion of BM25 and Dense retrieval
4. **Hybrid + Rerank** using a cross-encoder on top of fused candidates

The project goal is not only to identify the strongest system on average, but also to explain system behavior across query types, candidate coverage, efficiency, and rerank depth.

# 2 Dataset

We use the **MTEB StackOverflowDupQuestions** benchmark for duplicate question retrieval on Stack Overflow. The benchmark provides a document corpus, a query set, and pre-existing relevance judgements (qrels), making it well suited for controlled retrieval evaluation.

Table 1: Dataset Statistics

| Split | Documents | Queries |
|---|---|---|
| Train | 593,522 | 19,847 |
| Test | 89,470 | 2,992 |

The final experiments use the aligned **test** split for corpus, queries, and qrels. Each document is constructed by concatenating the question title and body into a single text field. Null fields are replaced with empty strings. Queries are retained only if they have corresponding relevance judgements.

The original proposal intended to restrict evaluation to Python, Java, and JavaScript questions. A metadata-based language filtering check was implemented, but the released benchmark schema did not expose a reliable language field such as `language`, `lang`, or `tags`. As a result, the final evaluation is performed on the full test split. This yields a more difficult retrieval setting and therefore a stronger benchmark for the system comparison.

# 3 Methodology

## 3.1 System Overview

The final system consists of four retrieval configurations:

1. BM25 baseline

2. Dense baseline
3. Hybrid fusion using RRF
4. Hybrid fusion followed by cross-encoder reranking

**Query → BM25 top-100 + Dense top-100 → RRF fusion → Cross-Encoder rerank top-20 → Final top-10**

Figure 1: Hybrid Retrieve-and-Rerank Pipeline

### 3.2 BM25 Retrieval

The sparse retriever is implemented with Pyserini on top of Lucene. The corpus is serialized to JSONL and indexed with BM25. This component is especially strong when exact lexical overlap matters, such as error messages, exception names, API identifiers, or highly specific programming terminology.

### 3.3 Dense Retrieval

Dense retrieval uses the Sentence-Transformers model `all-MiniLM-L6-v2`. All corpus documents are encoded into 384-dimensional vectors and stored in a FAISS `IndexFlatIP`. Since query and document embeddings are L2-normalized, exact inner-product search is equivalent to cosine similarity search. This stage is intended to capture semantic relationships beyond direct token overlap.

### 3.4 Reciprocal Rank Fusion

Hybrid retrieval merges the BM25 and Dense ranked lists using Reciprocal Rank Fusion:

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)}$$

where $k = 60$ is the fusion constant and $R$ denotes the ranked lists being combined. Documents that rank highly in both BM25 and Dense receive the strongest fused scores.

### 3.5 Cross-Encoder Reranking

The reranking stage uses `cross-encoder/ms-marco-MiniLM-L-6-v2`. Unlike the bi-encoder, the cross-encoder jointly processes the query and candidate document and produces a more fine-grained relevance score. Because this stage is computationally more expensive, it is applied only to a limited fused candidate set. The final best reranked system uses a top-20 fused candidate pool, as determined by ablation.

### 3.6 Implementation Configuration

Table 2: Final Experiment Configuration

| Setting | Value |
|---|---|
| Random seed | 42 |
| Bi-encoder | `all-MiniLM-L6-v2` |
| Cross-encoder | `cross-encoder/ms-marco-MiniLM-L-6-v2` |
| BM25 candidate pool | 100 |
| Dense candidate pool | 100 |
| RRF constant | 60 |
| Shared MAP depth | 20 |
| Best rerank depth | 20 |
| Final user-facing cutoff | 10 |

## 4 Experimental Setup

The systems are evaluated on all 2,992 test queries.

The main ranking metrics are:

- **Precision@10**
- **Recall@10**
- **nDCG@10**
- **MAP**

Precision@10, Recall@10, and nDCG@10 are computed at cutoff 10. MAP is computed at a shared ranking depth of 20 for all systems, so that the comparison remains fair even when the reranker uses a shallower candidate set.

To assess whether the observed ranking improvements are statistically reliable, a one-sided Wilcoxon signed-rank test is applied to per-query nDCG@10 scores.

## 5 Results

### 5.1 Main Evaluation Results

Table 3: Main Evaluation Results

| System | Precision@10 | Recall@10 | MAP | nDCG@10 |
|---|---|---|---|---|
| BM25 | 0.663068 | 0.221738 | 0.309820 | 0.692774 |
| Dense | 0.665441 | 0.222582 | 0.306795 | 0.697388 |
| Hybrid (RRF) | **0.721491** | **0.241327** | **0.350161** | **0.747357** |
| Hybrid + Rerank (top-20, BEST) | 0.704813 | 0.235739 | 0.343802 | 0.729364 |

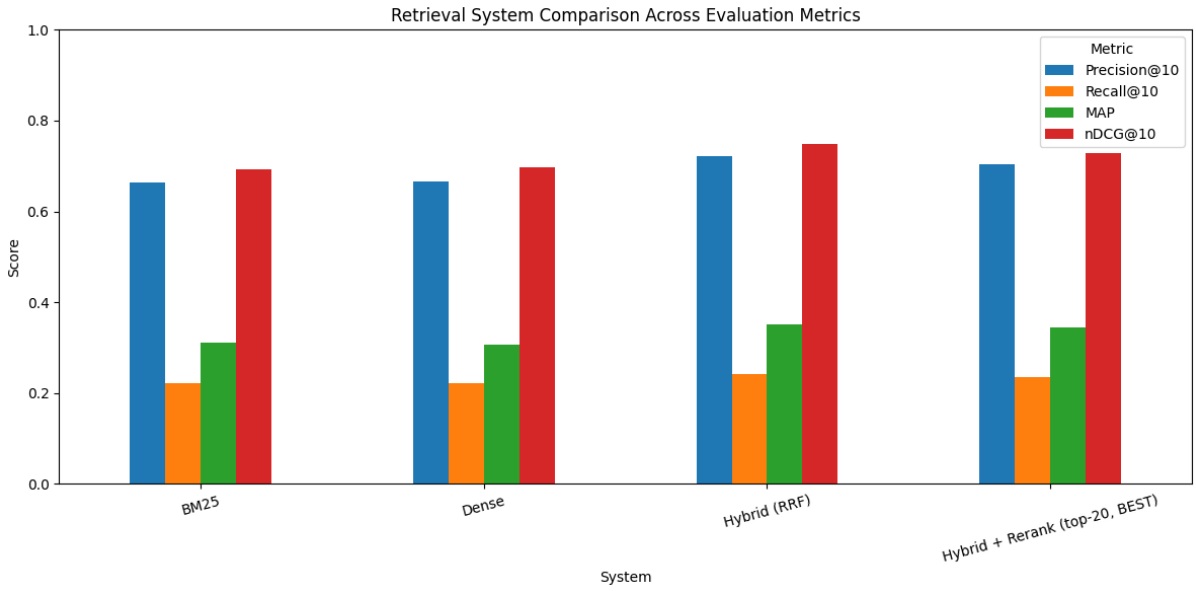
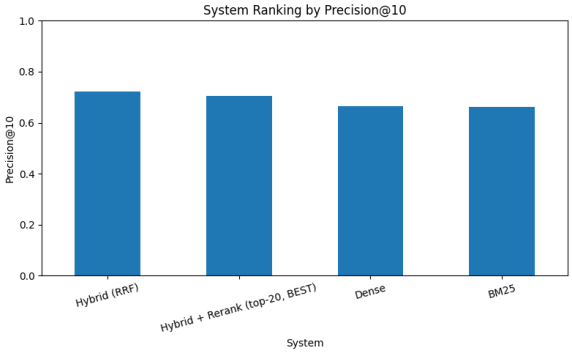



Figure 2: Comparison across Precision@10, Recall@10, MAP, and nDCG@10

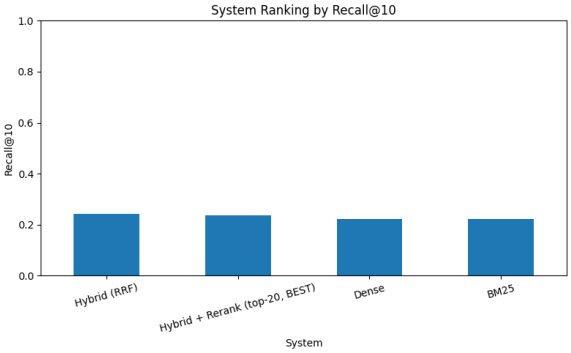
Hybrid (RRF) is the strongest overall system. It outperforms BM25, Dense retrieval, and the best reranked configuration across all aggregate metrics.

## 5.2 Per-Metric Visual Comparison

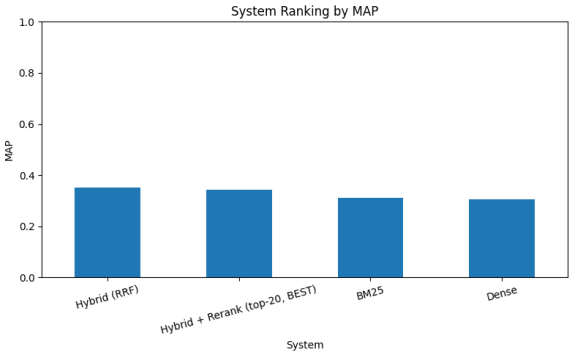
To complement the main evaluation table, Figures 3a–3d show the ranking of systems under each individual metric.



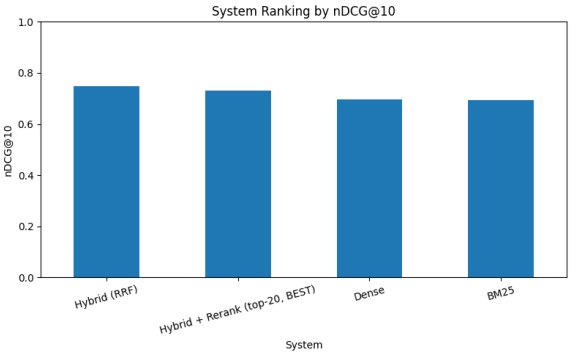

(a) Precision@10




(b) Recall@10




(c) MAP




(d) nDCG@10

Figure 3: Per-metric comparison of the four retrieval systems. Hybrid (RRF) ranks first across all four aggregate metrics.

### 5.3 Improvement Relative to BM25

Table 4: Absolute Improvement Relative to BM25

| **System** | ΔP@10 | ΔR@10 | ΔMAP | ΔnDCG@10 |
|---|---|---|---|---|
| Dense | +0.002373 | +0.000844 | -0.003025 | +0.004614 |
| Hybrid (RRF) | **+0.058423** | **+0.019589** | **+0.040341** | **+0.054583** |
| Hybrid + Rerank (top-20, BEST) | +0.041745 | +0.014001 | +0.033982 | +0.036590 |

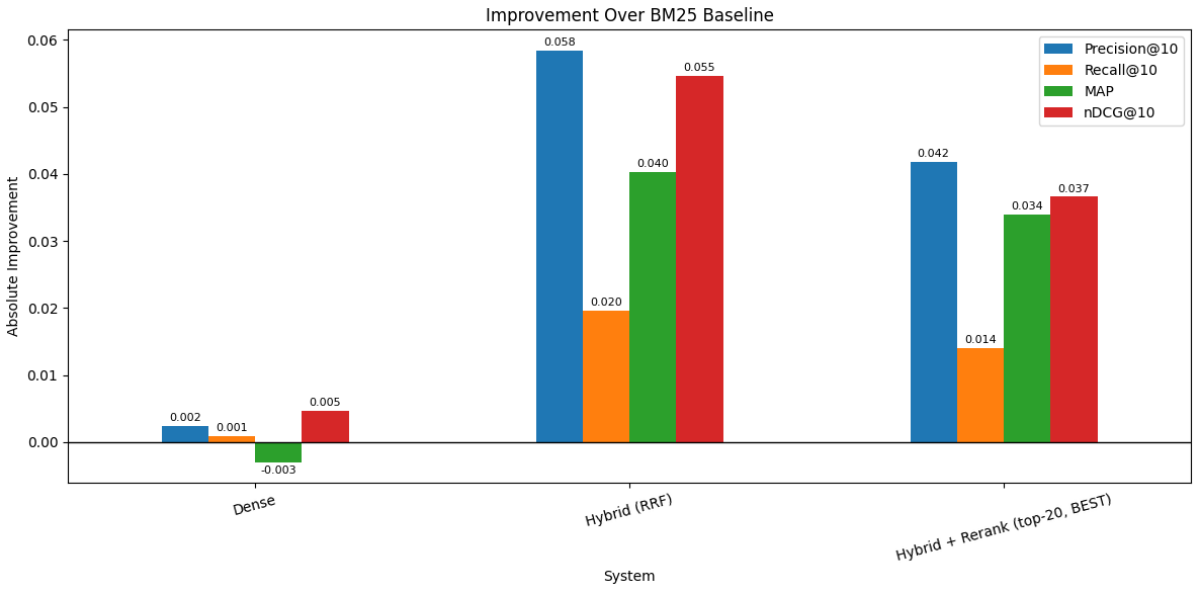



Figure 4: Absolute improvement relative to the BM25 baseline

The hybrid fused system yields the largest gains over BM25 across all metrics. Dense retrieval is slightly better than BM25 on Precision@10, Recall@10, and nDCG@10, but slightly lower on MAP in this setup.

### 5.4 Statistical Significance

Table 5: Wilcoxon Signed-Rank Tests on Per-Query nDCG@10

| Comparison | p-value | Conclusion |
|---|---|---|
| Hybrid (RRF) vs BM25 | $< 10^{-6}$ | Significant |
| Hybrid + Rerank vs BM25 | $< 10^{-6}$ | Significant |
| Hybrid (RRF) vs Hybrid + Rerank | $< 10^{-6}$ | Significant |

All three comparisons are significant. Hybrid (RRF) not only outperforms BM25, but also significantly outperforms the best reranked configuration on per-query nDCG@10.

## 6 Analysis

### 6.1 Rerank Depth Ablation

Table 6: Rerank Depth Ablation

| System | Precision@10 | Recall@10 | MAP | nDCG@10 |
|---|---|---|---|---|
| Hybrid + Rerank (top-20) | **0.704813** | **0.235739** | **0.343802** | **0.729364** |
| Hybrid + Rerank (top-50) | 0.676337 | 0.226206 | 0.318898 | 0.705707 |
| Hybrid + Rerank (top-100) | 0.663870 | 0.222036 | 0.307924 | 0.695400 |

The reranker performs best when applied to the top-20 fused candidates. Performance degrades as the rerank depth increases from 20 to 50 to 100. A plausible explanation is that the cross-encoder, trained on MS MARCO web-search data, is less well matched to deeper and noisier technical candidate pools from Stack Overflow.

### 6.2 Candidate Recall

Table 7: Average Candidate Recall@100

| System | Recall@100 |
|---|---|
| BM25 | 0.732 |
| Dense | 0.713 |
| Hybrid (RRF) | **0.795** |

The hybrid fused candidate pool achieves the highest Recall@100. This helps explain why Hybrid (RRF) performs best overall: it places more relevant documents into the candidate pool before final ranking.

### 6.3 Per-Query Winner Analysis

Table 8: Percentage of Queries Won by Each System (using nDCG@10)

| System | % Queries Won |
|---|---|
| BM25 | 36.40 |
| Dense | 26.77 |
| Hybrid (RRF) | 19.25 |
| Hybrid + Rerank | 17.58 |

No single system dominates every query. BM25 wins the most individual queries, especially when strong exact lexical cues are present. However, Hybrid (RRF) remains the best system on average because it is more consistently strong across the full query set.

### 6.4 Query-Type Analysis

Table 9: MAP by Query Type (shared ranking depth 20)

| Category | BM25 | Dense | Hybrid (RRF) | Hybrid + Rerank |
|---|---|---|---|---|
| API / library-specific | 0.310 | 0.307 | **0.351** | 0.345 |
| Error-code / exception | 0.323 | 0.301 | **0.351** | 0.343 |
| How-to | 0.309 | 0.311 | **0.349** | 0.342 |
| Symptom-based | 0.276 | 0.295 | **0.319** | 0.315 |

This breakdown reinforces the project motivation. BM25 remains relatively strong on precise technical queries, especially error-code and exception queries. Dense retrieval is slightly more

helpful on symptom-based and how-to queries. Hybrid (RRF) combines both signals most effectively and achieves the strongest MAP across all categories.

### 6.5 Runtime Efficiency

Table 10: Average Query Latency on a 50-Query Sample

| System | Seconds per Query |
|---|---|
| BM25 | 0.0012 |
| Dense | 0.0182 |
| Hybrid (RRF) | 0.0227 |
| Hybrid + Rerank (top-20) | 0.0388 |

BM25 is by far the fastest system. Hybrid (RRF) offers the strongest effectiveness while remaining much cheaper than full reranking. The best reranked system is slower, but still feasible for interactive use in a demo setting.

## 7 Qualitative Analysis

The qualitative examples align with the quantitative findings and directly illustrate the vocabulary-gap problem.

### 7.1 Exact Technical Query

For the query *java null pointer exception in arraylist*, BM25 retrieves the relevant document `apositive_test_query158_00000` at rank 1. Hybrid (RRF) also retrieves it, but slightly lower. This is a clear example where exact lexical overlap is highly informative and BM25 is naturally strong.

### 7.2 Symptom-Based Query

For the query *website is white after running javascript*, BM25 returns several results that match surface words such as *white* and *website*, but these are not always semantically close to the intended problem. Dense retrieval and Hybrid (RRF) return results such as *JavaScript not working on website* and *Visual studio 2015 Javascript is white?*, which are qualitatively closer to the underlying intent. This reflects the central vocabulary-gap motivation of the project.

### 7.3 Interactive Demo

A Gradio demo was built so that all four retrieval systems can be compared interactively on free-text programming questions. The reranking option uses the best ablation setting (top-20), so the UI is aligned with the final reported configuration.

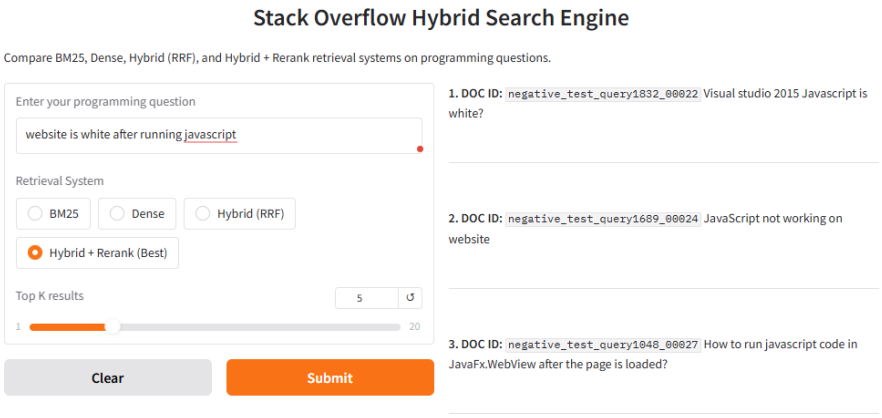



Figure 5: Interactive Gradio demo for BM25, Dense, Hybrid (RRF), and Hybrid + Rerank. The interface allows users to enter a free-text programming question, choose the retrieval system, and compare returned results interactively.

## 8 Conclusion

This project implemented and evaluated a hybrid retrieval pipeline for programming-question search on the MTEB StackOverflowDupQuestions benchmark. The results show that combining BM25 and dense retrieval through Reciprocal Rank Fusion is the most effective strategy in this setting. Hybrid (RRF) achieves the strongest performance on all main aggregate metrics and significantly outperforms BM25 as well as the best reranked configuration.

The deeper analysis clarifies why this happens. Hybrid fusion improves candidate coverage, lexical and semantic retrieval capture different forms of relevance, and a shallow reranking stage works better than a deeper one for this task. Overall, the project demonstrates that hybrid retrieval is a practical and effective way to reduce the vocabulary gap in technical question search.

## Acknowledgements

This project used the MTEB StackOverflowDupQuestions benchmark together with Pyserini, FAISS, Sentence-Transformers, and Gradio. All experiments were run in Google Colab.

## References

[1] Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. *Proceedings of the 32nd International ACM SIGIR Conference*, 758–759.

[2] Robertson, S. E. and Walker, S. (1994). Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval. *SIGIR '94*, 232–241.

[3] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP-IJCNLP 2019*.

[4] Nogueira, R. and Cho, K. (2019). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

[5] Muennighoff, N. et al. (2022). MTEB: Massive text embedding benchmark. *arXiv preprint arXiv:2210.07316*.

[6] Johnson, J., Douze, M., and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.